

DEPLOY_TONIGHT

Deploy It Live Tonight — NetworkPeer (GigGrid)

Verified on this machine (Aug 11, ~03:25): - Backend: `typecheck` , `lint` , `build` , unit tests  - **Funded atomic-acceptance E2E**  — create job → fund → signed webhook → POSTED → two workers race → exactly one `200` / one `409` , job `ASSIGNED` (run against local `networkpeer_e2e` DB, no external services) - Local infra up: PostGIS (`localhost:5433` , 39 migrations applied), Redis (`localhost:6379`) - Frontend dev server already running on `http://localhost:8080` (vite, HMR, points to `localhost:3000/api/v1`)

 The API on port 3000 was running 4-day-old code (`/api/v1/live` 404'd). It was restarted to fresh code, but background processes started from a shell here get killed, so **you must start it in your own terminal** (Step 1). Until then the frontend will show connection errors.

Option A — Live demo on this machine (~15 minutes, no accounts) — RECOMMENDED

Step 1: Start the API (keep this terminal open)

```
cd /Users/adityasharma/Desktop/NETWORKPEER/NetworkPeer-main
npm run build && node dist/index.js
```

Verify: `curl http://localhost:3000/api/v1/live` →
`{"success":true,...}`

Step 2: Confirm the frontend

Open `http://localhost:8080` in a normal browser and in a private/incognito window. If the dev server is not running: `cd NetworkPeer-platform-main && npm run dev` (it binds port 8080).

Step 3: Provision admin + verify the worker (one-time, run in another terminal)

First sign up the client (normal window) and worker (private window) through the UI (OTP code is echoed on the verify screen — dev mode only). Then:

```
cd /Users/adityasharma/Desktop/NETWORKPEER/NetworkPeer-main
docker exec networkpeer-postgis psql -U postgres -d networkpeer -
f scripts/provision-admin.sql
```

Then verify the worker (replace the phone numbers with the ones you used):

```
docker exec networkpeer-postgis psql -U postgres -d networkpeer \
--set=admin_phone="+1XXXXXXXXXX" \
--set=worker_phone="+1XXXXXXXXXX" <<'SQL'
SELECT public.admin_set_worker_verification(
  (SELECT id FROM public.users WHERE phone_number =
    :admin_phone'),
  (SELECT id FROM public.users WHERE phone_number =
    :worker_phone'),
  'VERIFIED', TRUE, 'Verified for live demonstration');
SQL
```

Step 4: Run the demo (follow docs/EXECUTIVE_PRESENTATION_AND_DEPLOYMENT_GUIDE.md §4)

Client creates job → Fund escrow → **instead of Stripe CLI**, settle funding with:

```
node scripts/simulate-payment-webhook.mjs <operationId>
<providerReference>
```

The `<operationId>` / `<providerReference>` are shown by the UI after clicking **Fund escrow** (they're also in the API response). Refresh the client job → status becomes `POSTED`. Then continue: worker accepts (PostGIS search), live task statuses, submit.

Known limitation: evidence upload (Step C) needs S3. Tonight either (a) skip that step, or (b) create a real bucket (~10 min, see below), or (c) ask to wire a local MinIO endpoint into the code.

Option B — Public URL so others can open it (+10 min)

Only needed if the presentation is on a different machine/network. Requires installing cloudflared:

```
brew install cloudflared
```

Then two terminals (URLs are random per run and change if you restart):

```
cloudflared tunnel --url http://localhost:3000    # gives API
              URL, e.g. https://xxx.trycloudflare.com
cloudflared tunnel --url http://localhost:8080    # gives
              FRONTEND URL
```

Then point the frontend at the API tunnel and the API at the frontend origin:

```
# Terminal A: rebuild frontend with the API tunnel URL
cd NetworkPeer-platform-main
VITE_API_BASE_URL="https://API-TUNNEL-URL/api/v1"
VITE_API_PREFIX=/api/v1 npm run dev
# (dev reads env at startup; add them to .env.local if it must
  survive restarts)

# Terminal B: add the frontend tunnel origin to CORS and restart
  the API
# add: CORS_ORIGINS=http://localhost:8080,https://FRONTEND-
  TUNNEL-URL → NetworkPeer-main/.env, then restart API
```

Caveats: keep the laptop awake; don't restart the tunnels mid-demo (URLs change); CORS must match exactly.

Option C — Real cloud deployment (NOT tonight — 2–4 h, needs accounts)

Follow `docs/EXECUTIVE_PRESENTATION_AND_DEPLOYMENT_GUIDE.md` §2 (Railway/Render + Vercel + AWS + Stripe test mode). Before anything:

1. **Commit + push the current work first** — `git status` shows many uncommitted changes; Git-based deploys would build the old commit. (Not done automatically.)

2. Generate real secrets: `openssl rand -hex 48 x3, -hex 32 x6` (guide §2.2).
 3. Create S3 bucket (versioning on, encryption on, public access blocked, CORS per §2.6), Stripe test keys + webhook, managed Postgres (PostGIS) + Redis TLS.
 4. Deploy API + worker services, run `npm run migrate` + `scripts/provision-app-role.sql` with the migration-owner URL **before** runtime roles get traffic.
 5. Deploy frontend on Vercel with `NITRO_PRESET=vercel`, Node 24, `VITE_API_BASE_URL`, `VITE_API_PREFIX`.
 6. Set backend `CORS_ORIGINS` to the exact Vercel origin, redeploy, verify `GET /api/v1/live`.
-

S3 for the evidence step (tonight, if you have an AWS account — ~10 min)

1. Create bucket, enable versioning, enable default encryption, enable all Block Public Access.
 2. Create an IAM user with programmatic keys; policy for `s3:PutObject`, `s3:GetObject` (HeadObject), `s3:PutObjectTagging`, `s3:GetBucketVersioning`, `s3:GetBucketPublicAccessBlock`, `s3:GetEncryptionConfiguration`, `s3:PutBucketCORS` on the bucket (the first two were wrongly named `GetPublicAccessBlock` / `GetBucketEncryption` before, and `PutBucketCORS` was missing).
 3. Add to `NetworkPeer-main/.env`: `AWS_REGION`, `AWS_ACCESS_KEY_ID`, `AWS_SECRET_ACCESS_KEY`, `AWS_S3_BUCKET`.
 4. Set bucket CORS for `http://localhost:8080` (§2.6 of the guide, with `FRONTEND_ORIGIN=http://localhost:8080`).
 5. Restart the API. Evidence upload in the demo then works end-to-end.
-

Demo recovery commands

```
curl -i http://localhost:3000/api/v1/live
curl -i http://localhost:3000/api/v1/health
```

```
docker exec networkpeer-postgis psql -U postgres -d networkpeer -  
c "SELECT filename, applied_at FROM schema_migrations  
ORDER BY filename DESC LIMIT 5;"  
docker logs --tail 50 networkpeer-postgis
```